COD 2023 - 2025 Sales

Elliot Dean
Office of the Chief Financial Officer | Office of the Assessor
City of Detroit

20 August 2025

Contents

Load Necessary Libraries

```
library(tidyverse)   # data wrangling & ggplot2
library(janitor)     # functions for examining and cleaning dirty data
library(haven)       # to read SPSS .sav files
library(lmtest)      # Breusch-Pagan test
library(car)         # vif()
library(ggpubr)      # Automatically print the regression equation for ggplot
library(spgwr)       # Functions for computing geographically weighted regressions
library(tidymodels)  # collection of packages for modeling and machine learning
library(xgboost)     # Extreme Gradient Boosting
library(bundle)      # Serialize Model Objects with a Consistent Interface
library(psych)       # group statistics, including correlations and factor analysis
library(kableExtra)  # let R Markdown generate a LaTeX-based PDF
library(broom)
library(olsrr)       # package for p-value backward elimination
library(MASS)        # Functions & datasets for Venables & Ripley
library(rlang)
library(gt)
```

Custom Functions

```
# build plotting data for each model from the rows actually used
mk_plot_df <- function(mod, label){
  idx <- as.integer(rownames(model.frame(mod)))
  out <- df[idx, , drop = FALSE]
```

```r
  out$.pred  <- fitted(mod)
  out$.truth <- model.response(model.frame(mod))
  out$model  <- label
  out
}

# ---- helper: build a simple recipe for one target ----
make_recipe <- function(data, target, predictors) {
  drop_cols <- setdiff(names(data), c(target, predictors))
  recipe(as.formula(paste(target, "~ .")), data = data) |>
    step_rm(all_of(drop_cols)) |>
    step_naomit(all_predictors(), all_outcomes()) |>
    step_zv(all_predictors()) |>
    step_dummy(all_nominal_predictors(), one_hot = TRUE)
  # (no normalization needed for OLS; keep it simple)
}

# ---- extra, scale-free metrics from fold-level predictions ----
smape_vec <- function(truth, estimate) {
  mean(200 * abs(estimate - truth) / (abs(truth) + abs(estimate)), na.rm = TRUE)
}
nrmse_sd_vec <- function(truth, estimate)
  100 * yardstick::rmse_vec(truth, estimate) / stats::sd(truth, na.rm = TRUE)

# ---- FIXED: resample (CV) aggregator ----
fold_metrics <- function(cv_obj, target_name) {
  preds <- collect_predictions(cv_obj) # has columns: id, .pred, ... + <target_name>

  preds %>%
    dplyr::group_by(id) %>%
    dplyr::summarise(
      RMSE    = yardstick::rmse_vec( truth = .data[[target_name]], estimate = .pred),
      MAE     = yardstick::mae_vec ( truth = .data[[target_name]], estimate = .pred),
      R2      = yardstick::rsq_vec ( truth = .data[[target_name]], estimate = .pred),
      `NRMSE%`= nrmse_sd_vec(       truth = .data[[target_name]], estimate = .pred),
      sMAPE   = smape_vec(          truth = .data[[target_name]], estimate = .pred),
      .groups = "drop"
    ) %>%
    dplyr::summarise(
      dplyr::across(c(RMSE, MAE, R2, `NRMSE%`, sMAPE),
                    list(mean = mean, sd = sd)),
      .groups = "drop"
    ) %>%
    dplyr::mutate(target = target_name, .before = 1)
}

# ---- compile pretty tables ----
tbl <- function(x, cap=NULL, digits=4){
  knitr::kable(x, format = "latex", booktabs = TRUE, digits = digits, caption = cap) |>
    kable_styling(latex_options = "hold_position")
}

# test-set metrics (single holdout) + scale-free add-ons
add_holdout_scaled <- function(last_fit_obj, target_name){
  mets  <- collect_metrics(last_fit_obj) %>%
             dplyr::select(.metric, .estimate) %>%
             tidyr::pivot_wider(names_from = .metric, values_from = .estimate)
```

```r
  preds <- collect_predictions(last_fit_obj)
  truth_col <- if (".obs" %in% names(preds)) ".obs" else target_name

  tibble::tibble(
    target   = target_name,
    RMSE     = mets$rmse,
    MAE      = mets$mae,
    R2       = mets$rsq,
    `NRMSE%` = nrmse_sd_vec(preds[[truth_col]], preds$.pred),
    sMAPE    = smape_vec   (preds[[truth_col]], preds$.pred)
  )
}
```

Importing and Organizing Data

```r
df  <- read_csv('../data/salesSFapr2023toMarch2025Updated.csv') |>
  rename_with(toupper)
```

Syntax Reference File - templateSyntaxECFs-aug25.sps

Organize Data I

```r
#    1. Price Categories
df <- df |>
  mutate(
    PRICECLASS = case_when(    # Lines 25 -> 34
      SALEPRICE < 48000                          ~ 1,
      SALEPRICE < 67000  & SALEPRICE >= 48000    ~ 2,
      SALEPRICE < 94000  & SALEPRICE >= 67000    ~ 3,
      SALEPRICE < 134000 & SALEPRICE >= 94000    ~ 4,
      SALEPRICE >= 134000                        ~ 5
    ),
    PRICECLASS = factor(       # Lines 36 -> 41
      PRICECLASS,
      levels = 1:5,
      labels = c(
        "Below 48000", "48000 to 67000", "67000 to 94000", "94000 to 134000", "134000+"
      )
    )
  )

df <- df |>
  #    2. Filter to Relevant Styles    Lines 60 -> 67
  filter(SRESB_STYLE %in% c("SINGLE FAMILY", "1/2 DUPLEX", "ROW HOUSE")) |>
  #    3. Deduplicate Records    Lines 93 -> 122
  arrange(PARCELNO, SALEPRICE, SALEDATE) |>
  group_by(PARCELNO, SALEPRICE, SALEDATE) |>
  mutate(
    PRIMARYFIRST = row_number() == 1,
    PRIMARYLAST  = row_number() == n()
  ) |>
```

```
  ungroup() |>
  filter(PRIMARYLAST) |>
  #    4. Exclude Assessment-Equal Sales    Lines 129 -> 140
  mutate(
    PRICEISASMT_RATIO  = AVWHENSOLD / SALEPRICE,
    PRICEISASMT_RATIO2 = if_else(between(PRICEISASMT_RATIO, .90, 1.10), 1, PRICEISASMT_RATIO),
    PRICEISASMT        = if_else(PRICEISASMT_RATIO == 1, 0L, 1L)
  ) |>
  filter(PRICEISASMT == 1) |>
  #    5. Outlier Removal    Lines 71 -> 87
  mutate(
    OUT = case_when(
      STOTALSQFT      < 400   ~ 1L,
      SRESB_FLOORAREA < 400   ~ 1L,
      SALEPRICE       < 10000 ~ 1L,
      TRUE                    ~ 0L
    )
  ) |>
  filter(OUT == 0) |>
  #    6. Story-Style Dummies    9. Year-Built Era  Lines 225 -> 330
  mutate(
    ONESTORY            = as.integer(SRESB_STYHGT <= 2),
    TWOSTORY            = as.integer(SRESB_STYHGT > 2 & SRESB_STYHGT <= 5),
    SINGLE_FAMILY1STORY = as.integer(SRESB_STYLE == "SINGLE FAMILY" & ONESTORY == 1),
    SINGLE_FAMILY2STORY = as.integer(SRESB_STYLE == "SINGLE FAMILY" & TWOSTORY == 1),
    HALF_DUPLEX1STORY   = as.integer(SRESB_STYLE == "1/2 DUPLEX" & ONESTORY == 1),
    HALF_DUPLEX2STORY   = as.integer(SRESB_STYLE == "1/2 DUPLEX" & TWOSTORY == 1),
    ERABUILT = case_when(
      SRESB_YEARBUILT <  1910                            ~ 1,
      SRESB_YEARBUILT <  1930 & SRESB_YEARBUILT >= 1910 ~ 2,
      SRESB_YEARBUILT <  1946 & SRESB_YEARBUILT >= 1930 ~ 3,
      SRESB_YEARBUILT <  1961 & SRESB_YEARBUILT >= 1946 ~ 4,
      SRESB_YEARBUILT <= 1990 & SRESB_YEARBUILT >= 1961 ~ 5,
      SRESB_YEARBUILT >  1990                            ~ 6
    ),
    YB_PRE_1929     = as.integer(ERABUILT %in% c(1,2)),
    YB_1930_TO_1945 = as.integer(ERABUILT == 3),
    YB_1946_TO_1960 = as.integer(ERABUILT == 4),
    YB_POST_1961    = as.integer(ERABUILT %in% c(5,6))
  ) |>
  #    7. Condition Dummies    Lines 334 -> 386
  mutate(
    EXCELLENT          = as.integer(SCOND == 0),
    VERY_GOOD          = as.integer(SCOND == 1),
    GOOD               = as.integer(SCOND == 2),
    AVERAGE            = as.integer(SCOND == 3),
    FAIR               = as.integer(SCOND == 4),
    POOR               = as.integer(SCOND == 5),
    VERY_POOR_UNSOUND  = as.integer(SCOND %in% c(6,7)),
    TWO_PLUS_BATHS     = as.integer(SRESB_FULLBATHS >= 2),
    TWO_BATHS          = as.integer(SRESB_FULLBATHS == 2),
    POWDER_ROOM        = as.integer(SRESB_HALFBATHS >= 1),
    ONE_BATH           = as.integer(SRESB_FULLBATHS == 1),
    TOTAL_BATHS        = as.integer(SRESB_FULLBATHS + 0.5 * SRESB_HALFBATHS),
    FIREPLACE          = as.integer(SRESB_FIREPLACE >= 1)
  ) |>
  #    8. Basement & Garage    Lines 393 -> 473
  mutate(
```

```
  PCTCRAWL       = SRESB_CRAWSPACE / SRESB_GROUNDAREA,
  PCTSLAB        = SRESB_SLABAREA / SRESB_GROUNDAREA,
  PCTBSMNT       = SRESB_BASEMENTAREA / SRESB_GROUNDAREA,
  CRAWL          = as.integer(PCTCRAWL > PCTBSMNT & PCTCRAWL > PCTSLAB),
  SLAB           = as.integer(PCTSLAB  > PCTCRAWL & PCTSLAB  > PCTBSMNT),
  GARAGE         = as.integer(SRESB_GARAGEAREA >= 200),
  ONE_CAR_GARAGE = as.integer(SREB_GARTYPE == 1),
  TWO_CAR_GARAGE = as.integer(SREB_GARTYPE == 2),
  GARAGESPACES   = case_when(
    SRESB_GARAGEAREA > 200 & SRESB_GARAGEAREA < 420 ~ 1,
    SRESB_GARAGEAREA > 419 & SRESB_GARAGEAREA < 600 ~ 2,
    SRESB_GARAGEAREA > 599 & SRESB_GARAGEAREA < 820 ~ 3,
    SRESB_GARAGEAREA > 819                          ~ 4
  ),
  LN_GARAGESPACES = log(GARAGESPACES)
) |>
#   9. Quality & Street Class    Lines 478 -> 544
mutate(
  QABOVE_AVG      = as.integer(SRESB_BLDGCLASS %in% c(3,4,5)),
  Q_AVG           = as.integer(SRESB_BLDGCLASS == 2),
  QBELOW_AVG      = as.integer(SRESB_BLDGCLASS %in% c(0,1)),
  ARTERIAL        = as.integer(RD_CLASS == "A31"),
  FORCED_AIR      = as.integer(SRESB_HEAT %in% c(0,1)),
  HOT_WATER       = as.integer(SRESB_HEAT == 2),
  ELEC_BASEBOARD  = as.integer(SRESB_HEAT == 3),
  ELEC_RADIANT    = as.integer(SRESB_HEAT == 4),
  RADIANT_FLOOR   = as.integer(SRESB_HEAT == 5),
  ELEC_WALL       = as.integer(SRESB_HEAT == 6),
  SPACE_HEATER    = as.integer(SRESB_HEAT == 7),
  WALL            = as.integer(SRESB_HEAT == 8),
  CENTRAL_AIR     = as.integer(SRESB_HEAT == 9),
  SIZE = case_when(
    SRESB_FLOORAREA <   830                              ~ 1,
    SRESB_FLOORAREA <   1018 & SRESB_FLOORAREA >= 830    ~ 2,
    SRESB_FLOORAREA <   1266 & SRESB_FLOORAREA >= 1018   ~ 3,
    SRESB_FLOORAREA <   1659 & SRESB_FLOORAREA >= 1266   ~ 4,
    SRESB_FLOORAREA >=  1659                             ~ 5
  ),
  SMALLEST = as.integer(SIZE == 1), SMALL = as.integer(SIZE == 2),
  AVG = as.integer(SIZE == 3),      LARGE = as.integer(SIZE == 4),
  LARGEST = as.integer(SIZE == 5)
) |>
#   10. Building Size & Lot Ratios    Lines 551 -> 593
mutate(
  RATIO                 = SESTTCV / SALEPRICE,
  SPPSF                 = SALEPRICE / SRESB_FLOORAREA,
  BASE_BLDGSIZE         = case_when(
                          SRESB_STYLE == "SINGLE FAMILY" ~ 1056,
                          SRESB_STYLE == "1/2 DUPLEX"    ~ 840,
                          TRUE                           ~ NA_real_
                        ),
  BASE_BLDGSIZE_RATIO   = SRESB_FLOORAREA / BASE_BLDGSIZE,
  LN_BASE_BLDGSIZE_RATIO = log(BASE_BLDGSIZE_RATIO),
  LANDRATIO             = STOTALSQFT / case_when(
                          SRESB_STYLE == "SINGLE FAMILY" ~ 4599,
                          SRESB_STYLE == "1/2 DUPLEX"    ~ 3180,
                          TRUE                           ~ NA_real_
                        ),
```

```
    LANDRATIO2             = pmin(LANDRATIO, if_else(SRESB_STYLE == "SINGLE FAMILY",4,3)),
    LN_LANDRATIO           = log(LANDRATIO2),
    SALEDATE       = as_date(SALEDATE),
    MONTHS         = interval(min(SALEDATE), SALEDATE) %/% months(1), # LINE 498
    MONTHS_1TO9    = case_when(MONTHS <= 9  ~ MONTHS, TRUE ~ 9),
    MONTHS_10TO16 = case_when(MONTHS <= 9  ~ 0, MONTHS <= 16 ~ MONTHS - 9, TRUE ~ 7),
    MONTHS_17TO24 = case_when(MONTHS <= 16 ~ 0, TRUE ~ MONTHS - 16),
    LN_SPPSF       = log(SPPSF),
    LN_PRICE       = log(SALEPRICE),
    LNSBLDSF       = log(SRESB_FLOORAREA), SBLDSFINT = SRESB_FLOORAREA - 432,
    LNSBLDSFINT    = log(SBLDSFINT),  # LINEs 650 -> 658
  )
```

Organize Data II

```
RATE1 <- 0.004818; RATE2 <- 0.016146; RATE3 <- 0.0; END_INDEX <- 1.17

df <- df |>
  #    11. Time Variables & Price Adjustment    Lines 520 -> 532
  mutate(
    PRICE_INDEX    = (1 + RATE1)^MONTHS_1TO9 * (1 + RATE2)^MONTHS_10TO16 *
      (1 + RATE3)^MONTHS_17TO24,
    TAF            = END_INDEX / PRICE_INDEX,
    TASP           = SALEPRICE * TAF,
    LN_TASP        = log(TASP),
    TASPPSF        = TASP / SRESB_FLOORAREA
  )
psych::describe(df$PRICE_INDEX)
```

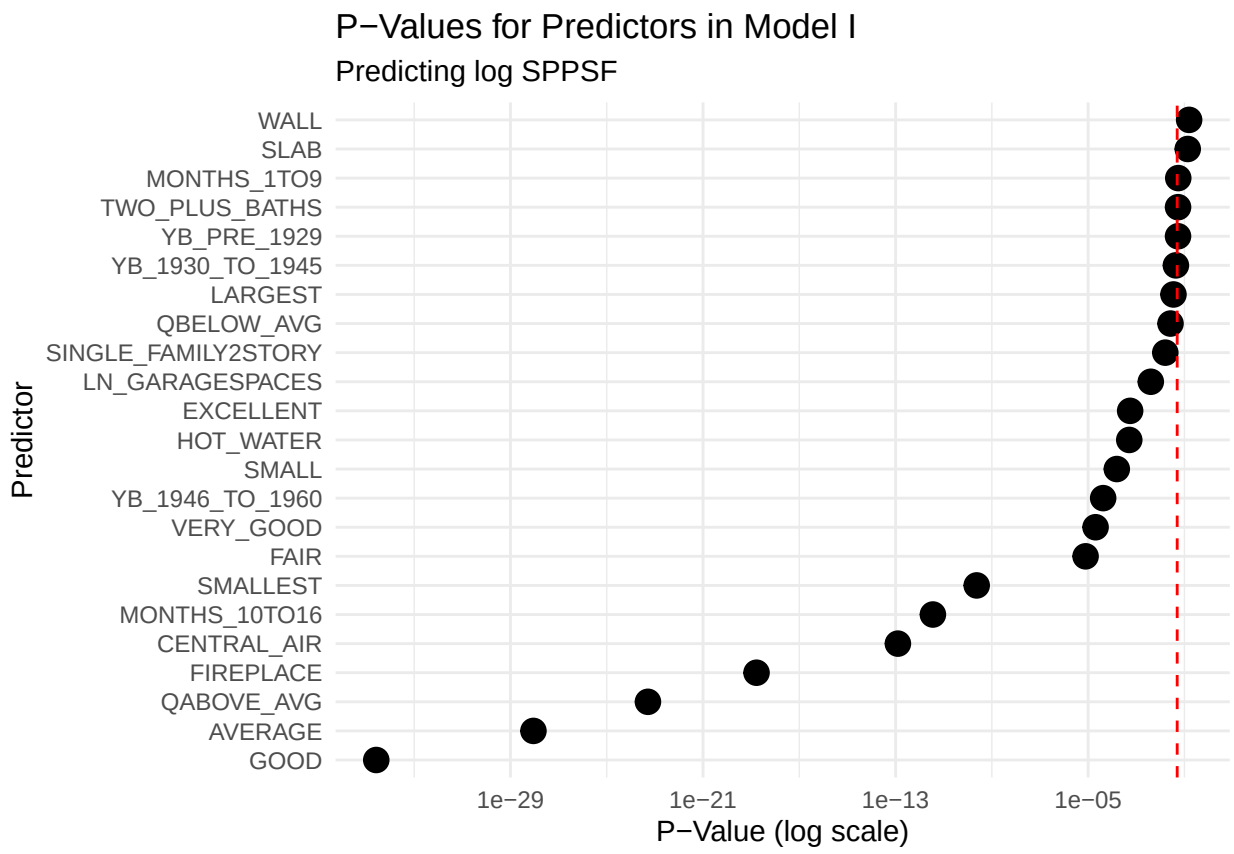
```
##    vars     n mean   sd median trimmed  mad min  max range  skew kurtosis se
## X1    1 14051  1.1 0.07    1.1     1.1 0.11   1 1.17  0.17 -0.14    -1.66  0
```

```
# graphing(df, MONTHS, PRICE_INDEX, 'cyan', 'Months', 'Price Index', 'Index By Month')
```

Create Models

```
# Initial full model ***
full_model_1 <- lm(LN_SPPSF ~ ARTERIAL + CRAWL + SLAB +
                   SINGLE_FAMILY1STORY + SINGLE_FAMILY2STORY +
                   HALF_DUPLEX1STORY + HALF_DUPLEX2STORY +
                   SMALLEST + SMALL + LARGE + LARGEST + AVG +
                   TWO_PLUS_BATHS + POWDER_ROOM + LN_GARAGESPACES + FIREPLACE +
                   EXCELLENT + VERY_GOOD + GOOD + AVERAGE + FAIR + POOR + VERY_POOR_UNSOUND +
                   QABOVE_AVG + QBELOW_AVG + Q_AVG +
                   YB_PRE_1929 + YB_1930_TO_1945 + YB_1946_TO_1960 + YB_POST_1961 +
                   HOT_WATER + ELEC_BASEBOARD + ELEC_WALL + WALL + CENTRAL_AIR +
                   RADIANT_FLOOR + SPACE_HEATER + ELEC_RADIANT +
                   MONTHS_1TO9 + MONTHS_10TO16 + MONTHS_17TO24,
                 data = df, na.action = na.omit)

# Perform backward stepwise regression
backward_model_1 <- stats::step(full_model_1, direction = "backward", trace = 0)
```

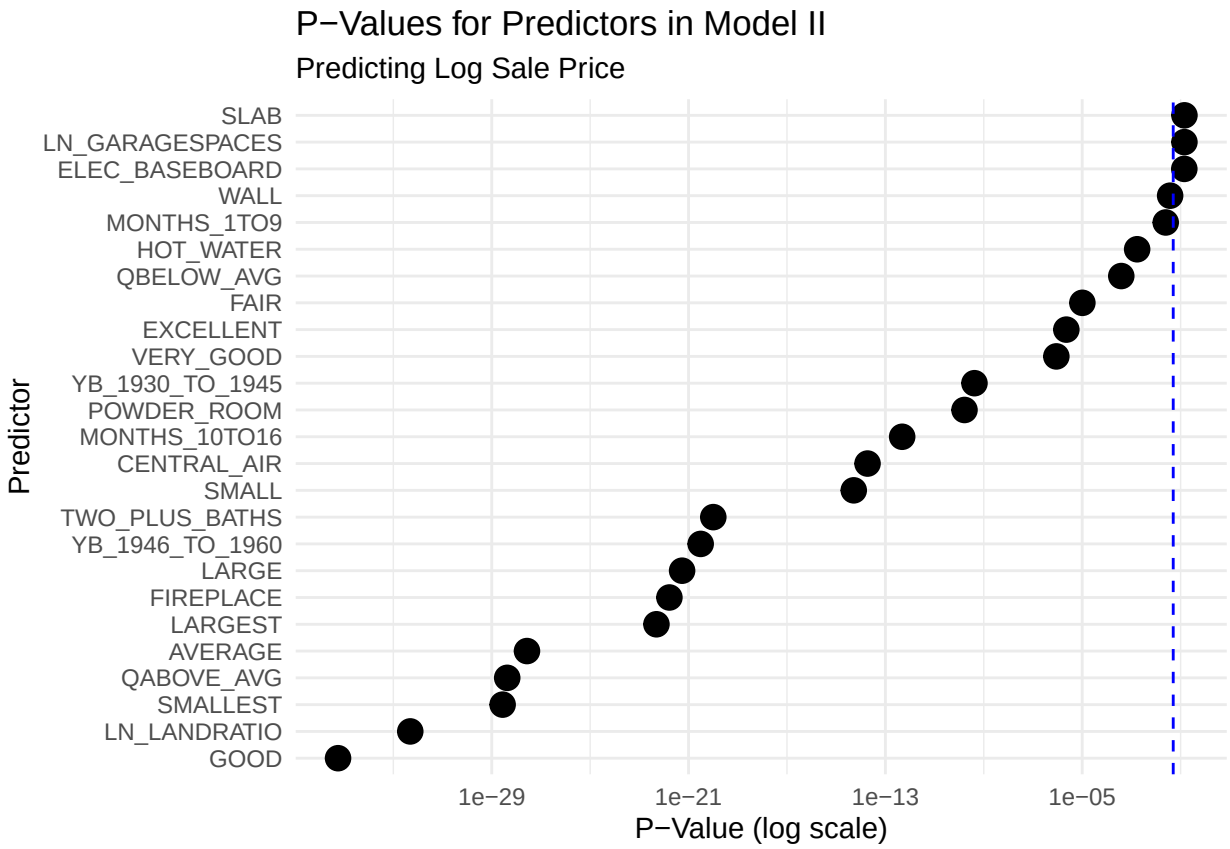
```r
# The tidy() function from the broom package converts the model output
# into a clean data frame with one row for each coefficient.
model_1_summary <- tidy(backward_model_1)

model_1_summary |>
  # We don't usually plot the p-value for the intercept
  filter(term != "(Intercept)") |>
  ggplot(aes(x = p.value, y = reorder(term, p.value))) +
  geom_point(size = 4) +
  # Add a vertical line for the common significance threshold (alpha = 0.05)
  geom_vline(xintercept = 0.05, linetype = "dashed", color = "red") +
  # Using a log scale for the x-axis is often essential for p-values,
  # as they can be clustered near zero.
  scale_x_log10() +
  labs(
    title = "P-Values for Predictors in Model I",
    subtitle = "Predicting log SPPSF",
    x = "P-Value (log scale)",
    y = "Predictor"
  ) +
  theme_minimal()
```



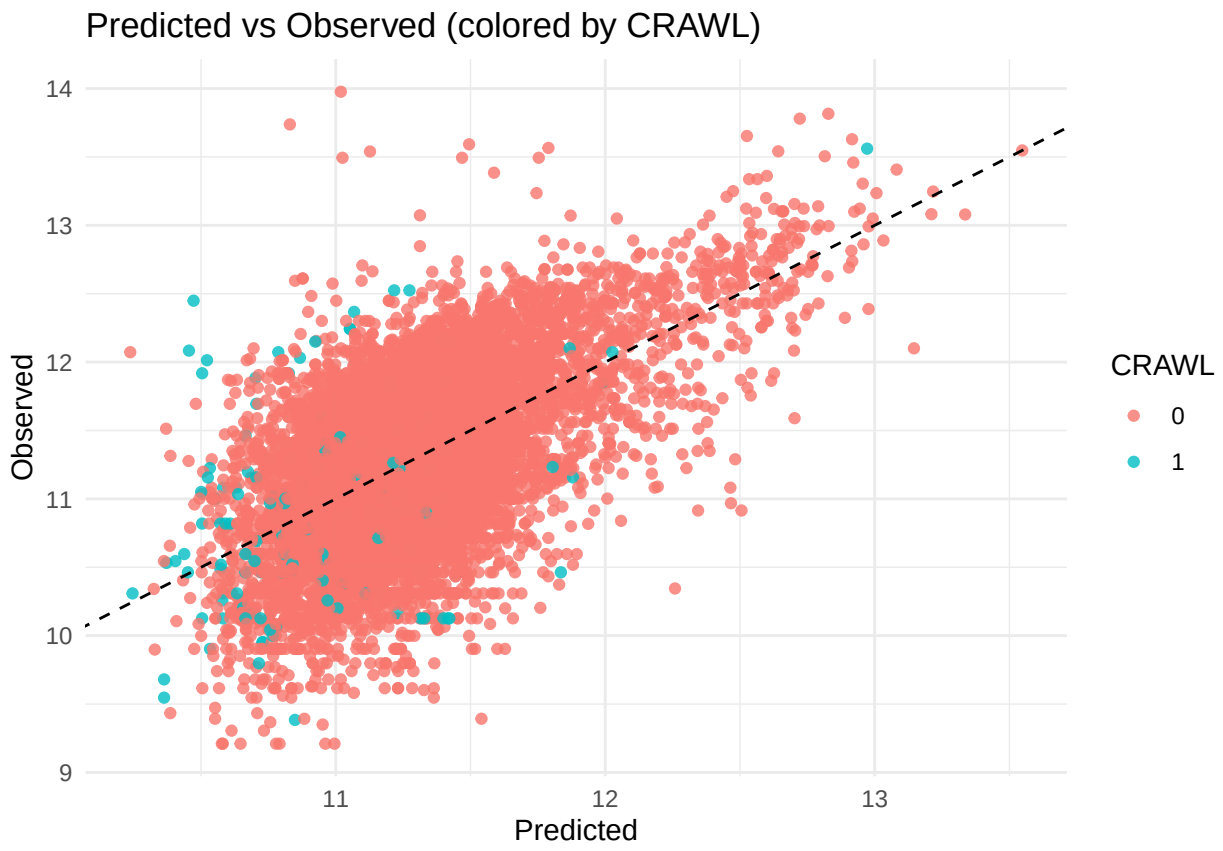

```r
# Initial full model *** Lines 762 -> 778
full_model_2 <- lm(LN_PRICE ~ ARTERIAL + CRAWL + SLAB + LN_LANDRATIO +
                     # LN_BASE_BLDGSIZE_RATIO +
                   SINGLE_FAMILY1STORY + SINGLE_FAMILY2STORY +
                   HALF_DUPLEX1STORY + HALF_DUPLEX2STORY +
                   SMALLEST + SMALL + LARGE + LARGEST + AVG +
                   TWO_PLUS_BATHS + POWDER_ROOM + LN_GARAGESPACES + FIREPLACE +
                   EXCELLENT + VERY_GOOD + GOOD + AVERAGE + FAIR + POOR + VERY_POOR_UNSOUND +
```

```
                    QABOVE_AVG + QBELOW_AVG + Q_AVG +
                    YB_PRE_1929 + YB_1930_TO_1945 + YB_1946_TO_1960 + YB_POST_1961 +
                    HOT_WATER + ELEC_BASEBOARD + ELEC_WALL + WALL + CENTRAL_AIR +
                    RADIANT_FLOOR + SPACE_HEATER + ELEC_RADIANT +
                    MONTHS_1TO9 + MONTHS_10TO16 + MONTHS_17TO24,
                  data = df, na.action = na.omit)

# Perform backward stepwise regression
backward_model_2 <- stats::step(full_model_2, direction = "backward", trace = 0)
```

```
# The tidy() function from the broom package converts the model output
# into a clean data frame with one row for each coefficient.
model_2_summary <- tidy(backward_model_2)

# rows actually used by lm (after na.omit)
used_idx2 <- as.integer(rownames(model.frame(backward_model_2)))

plot_df2        <- df[used_idx2, , drop = FALSE]
plot_df2$.pred  <- fitted(backward_model_2)
plot_df2$.truth <- model.response(model.frame(backward_model_2)) # LN_PRICE used by lm

model_2_summary |>
  # We don't usually plot the p-value for the intercept
  filter(term != "(Intercept)") |>
  ggplot(aes(x = p.value, y = reorder(term, p.value))) +
  geom_point(size = 4) +
  # Add a vertical line for the common significance threshold (alpha = 0.05)
  geom_vline(xintercept = 0.05, linetype = "dashed", color = "blue") +
  # Using a log scale for the x-axis is often essential for p-values,
  # as they can be clustered near zero.
  scale_x_log10() +
  labs(
    title = "P-Values for Predictors in Model II",
    subtitle = "Predicting Log Sale Price",
    x = "P-Value (log scale)",
    y = "Predictor"
  ) +
  theme_minimal()
```




```
ggplot(plot_df2, aes(x = .pred, y = .truth, color = factor(CRAWL))) +
  geom_point(alpha = 0.8) +
  geom_abline(slope = 1, intercept = 0, linetype = 2) +
  labs(title = "Predicted vs Observed (colored by CRAWL)",
       x = "Predicted", y = "Observed", color = "CRAWL") +
  theme_minimal()
```



```
# ============================================================
# Linear-model assessment + comparison (tidymodels)
# ============================================================

set.seed(123)

# ---- USER INPUTS ----
target1   <- "LN_SPPSF"          # e.g. log-price
target2   <- "LN_PRICE"       # e.g. an index
predictors <- c('CRAWL', 'SLAB', 'LN_LANDRATIO', 'ARTERIAL',
                'SINGLE_FAMILY1STORY', 'SINGLE_FAMILY2STORY',
                'HALF_DUPLEX1STORY', 'HALF_DUPLEX2STORY',
                'SMALLEST', 'SMALL', 'LARGE', 'LARGEST', 'AVG',
                'TWO_PLUS_BATHS', 'POWDER_ROOM', 'LN_GARAGESPACES', 'FIREPLACE',
                'EXCELLENT', 'VERY_GOOD', 'GOOD', 'AVERAGE', 'FAIR', 'POOR', 'VERY_POOR_UNSOUND',
                'QABOVE_AVG', 'QBELOW_AVG', 'Q_AVG',
                'YB_PRE_1929', 'YB_1930_TO_1945', 'YB_1946_TO_1960', 'YB_POST_1961',
                'HOT_WATER', 'ELEC_BASEBOARD', 'ELEC_WALL', 'WALL', 'CENTRAL_AIR',
                'RADIANT_FLOOR', 'SPACE_HEATER', 'ELEC_RADIANT',
                'MONTHS_1TO9', 'MONTHS_10TO16', 'MONTHS_17TO24')

# ---- keep only rows present for BOTH targets & predictors ----
keep_cols <- intersect(names(df), c(target1, target2, predictors))
stopifnot(all(c(target1, target2) %in% keep_cols))
d0 <- df[, keep_cols, drop = FALSE]

# guard against non-finite targets (e.g., log of non-positive)
is_finite_both <- is.finite(d0[[target1]]) & is.finite(d0[[target2]])
```

```r
d0 <- d0[is_finite_both, , drop = FALSE]

# ---- shared split & folds (same rows for both models) ----
split <- initial_split(d0, prop = 0.8)    # no stratification since targets differ
train <- training(split)
test  <- testing(split)
folds <- vfold_cv(train, v = 10)

rec1 <- make_recipe(train, target1, predictors)
rec2 <- make_recipe(train, target2, predictors)

spec_lm <- linear_reg() |> set_engine("lm")

wf1 <- workflow() |> add_recipe(rec1) |> add_model(spec_lm)
wf2 <- workflow() |> add_recipe(rec2) |> add_model(spec_lm)

# ---- fit resamples (CV) & holdout (test) for each target ----
cv1 <- fit_resamples(wf1, folds, metrics = metric_set(rmse, mae, rsq),
                     control = control_resamples(save_pred = TRUE))
cv2 <- fit_resamples(wf2, folds, metrics = metric_set(rmse, mae, rsq),
                     control = control_resamples(save_pred = TRUE))

last1 <- last_fit(wf1, split, metrics = metric_set(rmse, mae, rsq))
last2 <- last_fit(wf2, split, metrics = metric_set(rmse, mae, rsq))

cv_sum1 <- fold_metrics(cv1, target1)
cv_sum2 <- fold_metrics(cv2, target2)

cv_compare <- bind_rows(cv_sum1, cv_sum2)
tbl(cv_compare,
    cap = "10-fold CV metrics (mean ± sd) including scale-free measures")
```

Table 1: 10-fold CV metrics (mean ± sd) including scale-free measures

target	RMSE_mean	RMSE_sd	MAE_mean	MAE_sd	R2_mean	R2_sd	NRMSE%_mean	NRMSE%_sd
LN_SPPSF	0.5174	0.0117	0.4191	0.0109	0.1196	0.0128	89.6006	1.5829
LN_PRICE	0.5189	0.0122	0.4190	0.0113	0.3321	0.0217	79.7058	1.7091

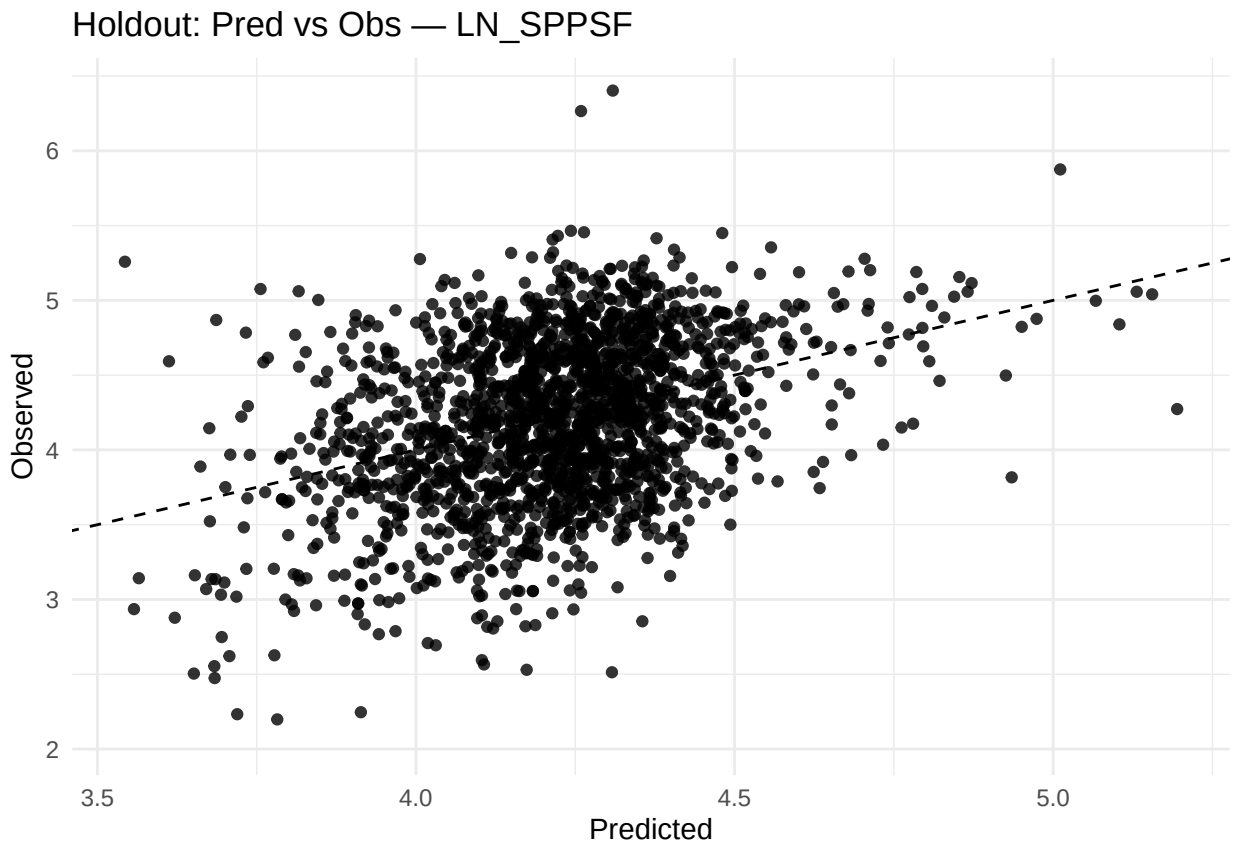
```r
test_compare <- bind_rows(
  add_holdout_scaled(last1, target1),
  add_holdout_scaled(last2, target2)
)
tbl(test_compare, cap = "Holdout test metrics with scale-free measures")
```

Table 2: Holdout test metrics with scale-free measures

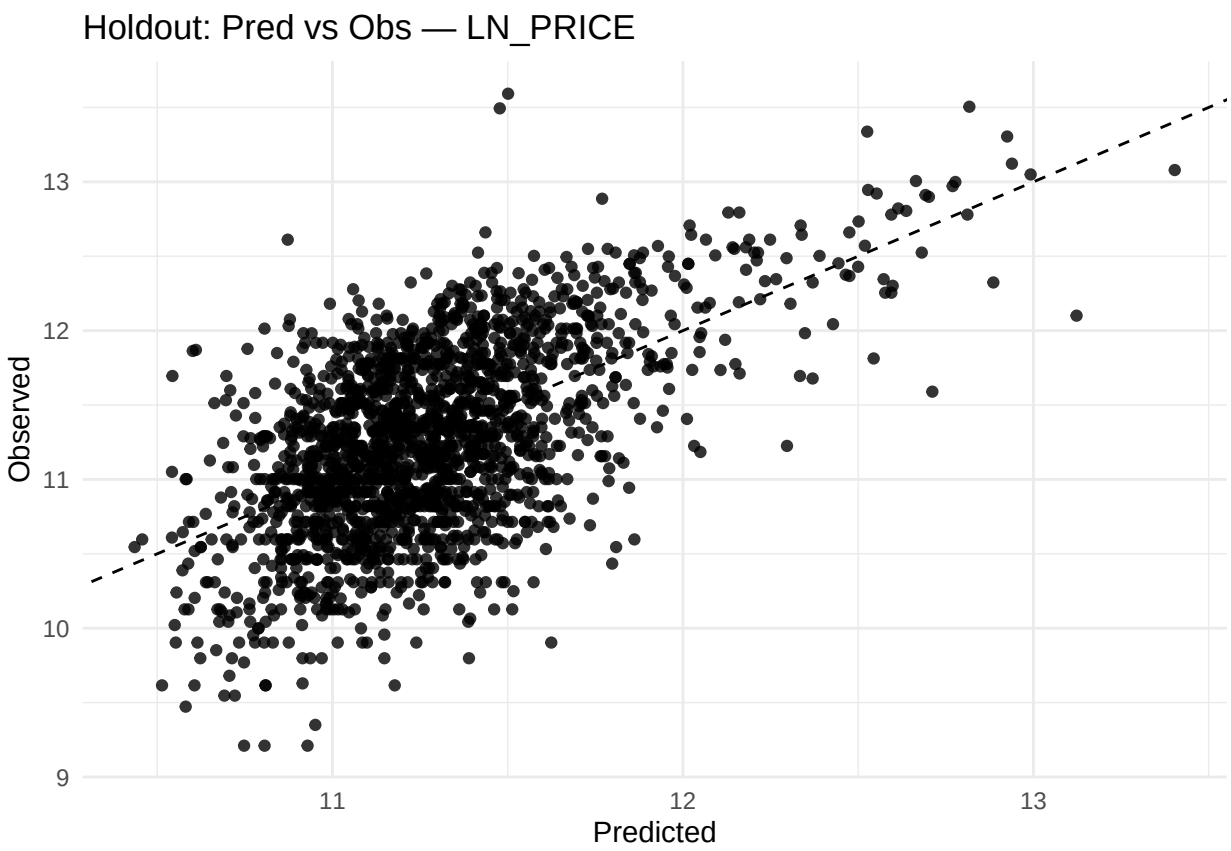
target	RMSE	MAE	R2	NRMSE%	sMAPE
LN_SPPSF	0.5112	0.4126	0.1441	88.3611	9.9931
LN_PRICE	0.5136	0.4145	0.3535	78.9023	3.6946

```r
# ---- (optional) quick plots for the PDF ----
# Predicted vs Observed for each target on the holdout
pdat1 <- collect_predictions(last1)
```

```r
truth1 <- if (".obs" %in% names(pdat1)) ".obs" else target1
if (truth1 != ".obs") {
  pdat1 <- dplyr::rename(pdat1, .obs = !!rlang::sym(target1))
}

ggplot(pdat1, aes(x = .pred, y = .obs)) +
  geom_point(alpha = 0.8) +
  geom_abline(slope = 1, intercept = 0, linetype = 2) +
  labs(title = paste("Holdout: Pred vs Obs -", target1),
       x = "Predicted", y = "Observed") +
  theme_minimal()
```




```r
pdat2 <- collect_predictions(last2)
truth2 <- if (".obs" %in% names(pdat2)) ".obs" else target2
if (truth2 != ".obs") {
  pdat2 <- dplyr::rename(pdat2, .obs = !!rlang::sym(target2))
}

ggplot(pdat2, aes(x = .pred, y = .obs)) +
  geom_point(alpha = 0.8) +
  geom_abline(slope = 1, intercept = 0, linetype = 2) +
  labs(title = paste("Holdout: Pred vs Obs -", target2),
       x = "Predicted", y = "Observed") +
  theme_minimal()
```


Holdout: Pred vs Obs — LN_PRICE
Observed
Predicted
13
12
11
10
9
11
12
13